Report Final Project

Owner: QUYETDT3

Project: CPU_32 bit of 45nm CMOS Digital Implementation

Date: Apr 16, 2026

1. Floorplan

a. IO Pad Analysis and Die Size Selection

At the beginning of the floorplanning stage, the first task was to determine the number of IO pads, because this directly affects the die size and the available core area.

Using the following command, a total of 83 pad-related instances were identified:

```
llength [get_db insts -if {.name == "*don't_touch*"}]
```

After reviewing the netlist and the design structure, one extra root clock buffer was found. This buffer was intentionally preserved for the CTS stage and therefore should not participate in the initial floorplan placement. For that reason, it was set to unplaced using:

```
set_db [get_db insts *root*] .place_status unplaced
```

After excluding this extra root clock buffer, the effective pad count became 82 pads. Among them, 4 pads are corner pads, which means the design contains 78 functional IO pads.

The final pad distribution was arranged as follows:

- 20 pads on the Left side
- 20 pads on the Right side
- 19 pads on the Top side
- 19 pads on the Bottom side

$$A_{core} = \frac{A_{std}}{0.65}$$

$$\frac{W_{core}}{H_{core}} = \frac{1260}{1300}$$

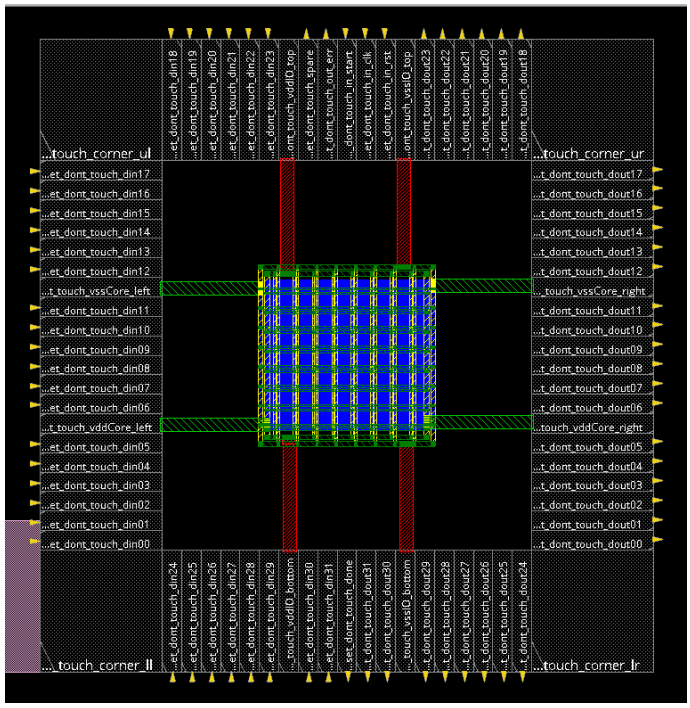
$$W_{core} = \sqrt{A_{core} \times \frac{1260}{1300}}$$

$$H_{core} = \sqrt{A_{core} \times \frac{1300}{1260}}$$

$$Margin_x = \frac{W_{die} - W_{core}}{2}$$

$$Margin_y = \frac{H_{die} - H_{core}}{2}$$

Based on a pad pitch of 40 and an offset of 250 from the die boundary on each side, the die size was calculated as 1260 in width and 1300 in height. This die shape was intentionally selected to keep the aspect ratio close to 1, which helps avoid a long and thin floorplan and supports a more balanced routing structure.



[Figure 1: IO pad distribution around the die boundary]

b. Standard Cell Area and Core Utilization

After determining the die size, the next step was to estimate the core area based on the total standard-cell area and the target utilization.

The total standard-cell area was calculated from the design database by excluding the IO and spare instances marked as "don't_touch":

```
set TOTAL_STD_CELL_AREA [expr [join [get_db [get_db insts -if {.name !=
"*don't_touch*"}] .area] +]]
```

From this calculation, the total standard-cell area was 64732.0158.

The target utilization was selected to be around 90%, with the goal of keeping the layout compact while still preserving enough room for routing resources, CTS insertion, and later timing optimization.

After applying the die size and utilization target, the resulting placement condition showed an actual utilization of approximately 90.35%. This value was considered acceptable because it keeps the design compact, but not overly dense. A compact floorplan is important for timing, while reasonable whitespace is still required for routability and buffer insertion.

```
@innovus 4> check_place
**Info: (IMPSP-307): Design contains fractional 17 cells.
Begin checking placement ... (start mem=1940.7M, init mem=1940.7M)
*info: Placed = 0
*info: Unplaced = 19526
Placement Density:90.35%(67736/74966)
Placement Density (including fixed std cells):90.35%(67736/74966)
```

[Figure 2: check_place / utilization screenshot]

c. Connectivity Violations

After running check_connectivity, two main violations remained in the design.

The first violation was related to the root clock buffer. This was expected because the root clock buffer had been intentionally kept unplaced and unrouted at the floorplan stage so that it could later be positioned properly for CTS. Therefore, this violation was not considered a real design error at this stage.

The second violation was related to the IO spare cells marked as "don't_touch". These cells do not belong to the functional logic of the CPU and are reserved for future use or spare strategy. Since they are intentionally preserved and not functionally connected, their disconnected status is also expected.

For this reason, both remaining connectivity violations were classified as acceptable stage-dependent violations, not final implementation errors.

```
Begin Summary
    2 Problem(s) (IMPVFC-98): Net has no global routing and no special routing.
    2 total info(s) created.
End Summary

End Time: Tue Apr  7 14:40:21 2026
Time Elapsed: 0:00:00.0

******** End: VERIFY CONNECTIVITY ********
  Verification Complete : 2 Viols.  0 Wrngs.
  (CPU Time: 0:00:00.2  MEM: 0.000M)
```

[Figure 3: check_connectivity result screenshot]

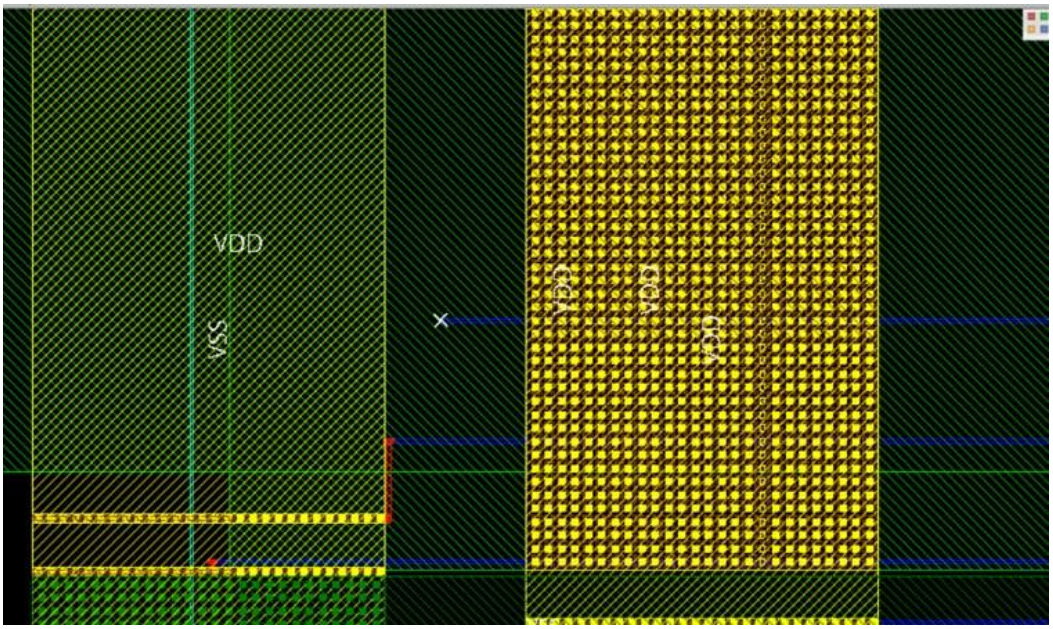
d. Observation on Dangling Wire and Spacing-Related Issues

Before finalizing the floorplan-related routing condition, the design previously reported four dangling wire violations.

The main reason was not that the nets were logically incomplete, but that the router stopped before reaching the target because continuing the wire would have created a spacing violation near an adjacent geometry or via. In other words, the issue initially appeared as a dangling wire, but its physical root cause was actually related to spacing limitation on the affected metal layer.

After checking the corresponding metal layer and reviewing the local spacing condition, the routing constraint was adjusted so that the router could legally continue the connection. As a result, the dangling wire issue was resolved without introducing a new spacing violation.

This debugging process confirmed that some early connectivity warnings were strongly related to physical routing conditions rather than to netlist connectivity itself.



[Figure 4: dangling wire violation screenshot]

2. Placement

a. Initial Placement Observation

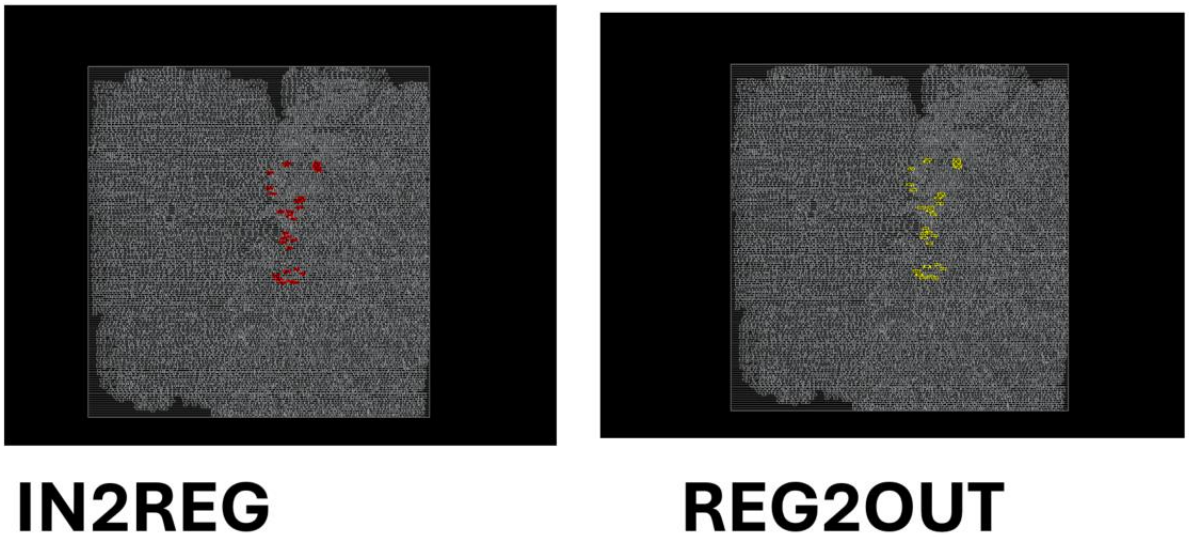
Before running placement optimization, the design showed a significant setup timing violation, with worst slack reaching approximately -15.229 ns. This result indicated that a normal placement result was not sufficient, and that the design needed a more guided placement strategy, especially around the timing-critical logic.

At this stage, the first step was to identify where the important cell groups were located, especially for in2reg, reg2out, sequential logic around the clock root, and the timing-critical arithmetic blocks inside the datapath.

From the initial observation, the registers involved in in2reg and reg2out were highly concentrated and should not be left scattered. To reduce timing distance, these cells should be kept near the center of the core.

```
+--------------------+---------+---------+---------+---------+
|     Setup mode     |   all   | reg2reg |reg2cgate| default |
+--------------------+---------+---------+---------+---------+
|           WNS (ns):| -15.229 | -15.229 | -0.661  | -0.776  |
|           TNS (ns):|-329.613 |-286.992 | -39.381 | -3.379  |
|    Violating Paths:|   252   |   130   |   105   |   19    |
|          All Paths:|  4527   |  4358   |   134   |   68    |
+--------------------+---------+---------+---------+---------+

+----------------+-------------------------------+------------------+
|                |              Real             |       Total      |
|    DRVs        +------------------+------------+------------------|
|                |  Nr nets(terms)  | Worst Vio  |  Nr nets(terms)  |
+----------------+------------------+------------+------------------+
|   max_cap      |      5 (5)       |   -0.083   |      5 (5)       |
|   max_tran     |    124 (4781)    |   -3.621   |    124 (4787)    |
|   max_fanout   |     47 (47)      |    -32     |     48 (48)      |
|   max_length   |      0 (0)       |     0      |      0 (0)       |
+----------------+------------------+------------+------------------+

Density: 86.353%
Routing Overflow: 0.00% H and 0.00% V
```

[Figure 5: Initial placement timing report / worst setup violation screenshot]



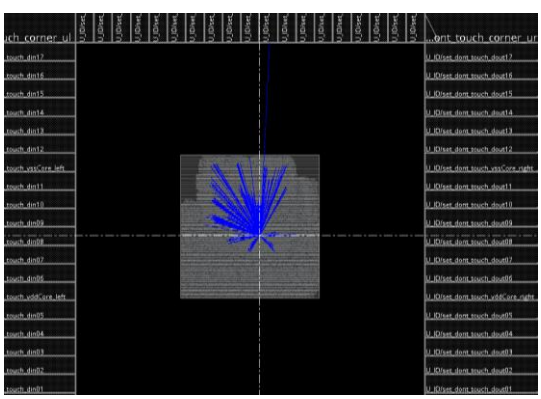
[Insert Figure 6: Highlighted in2reg / reg2out registers]

b. Root Clock Buffer Placement Strategy

The root clock buffer is one of the most important placement objects in this project because the final design requires a well-balanced clock tree distribution. For this reason, the root clock buffer should not be placed near the corner or at the edge of the chip. Instead, it should be located near the center of the sequential logic region, so that the clock sinks can be reached with better balance and reduced skew risk.

After checking the design hierarchy and the placement distribution, it was observed that a large portion of the sequential logic was concentrated around the ram_wrapper module. Therefore, placing the root clock buffer near the center of this region was considered the most reasonable physical solution.

This decision was not based only on geometric center, but also on the distribution of actual sequential sinks in the design.



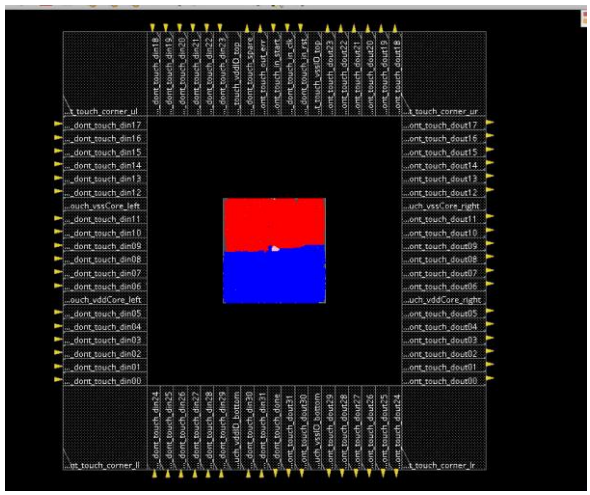
[Insert Figure 7: Root clock buffer location and surrounding sequential cells]

c. Placement Grouping and Boundary Strategy

From the area analysis, the two largest logic blocks in the design were ram_wrapper and ALU. Among the timing-critical paths, many of the worst paths were related to the ALU/div block. Therefore, a simple unconstrained placement was not sufficient. The design needed a region-based placement strategy to control the locations of the major timing-sensitive groups.

The final placement strategy was organized as follows.

- ram_wrapper: The ram_wrapper block occupies nearly half of the core area. In addition, the clock input of the design is located toward the top side of the chip. For these reasons, a region was assigned to place ram_wrapper in the upper part of the core, which gives a more natural placement structure and keeps it close to the root clock distribution area.
- Root Clock Buffer: The root clock buffer was placed near the center of the ram_wrapper region, so that the clock could fan out more effectively toward the large sequential population in this block.
- ALU/div: Based on the area report and timing behavior, the ALU/div block contributes strongly to the worst timing paths and occupies a large area, close to one-third of the core. Therefore, it was placed in a dedicated region below the ram_wrapper block. This arrangement reduces unnecessary distance between the major functional groups and creates a more structured datapath layout.



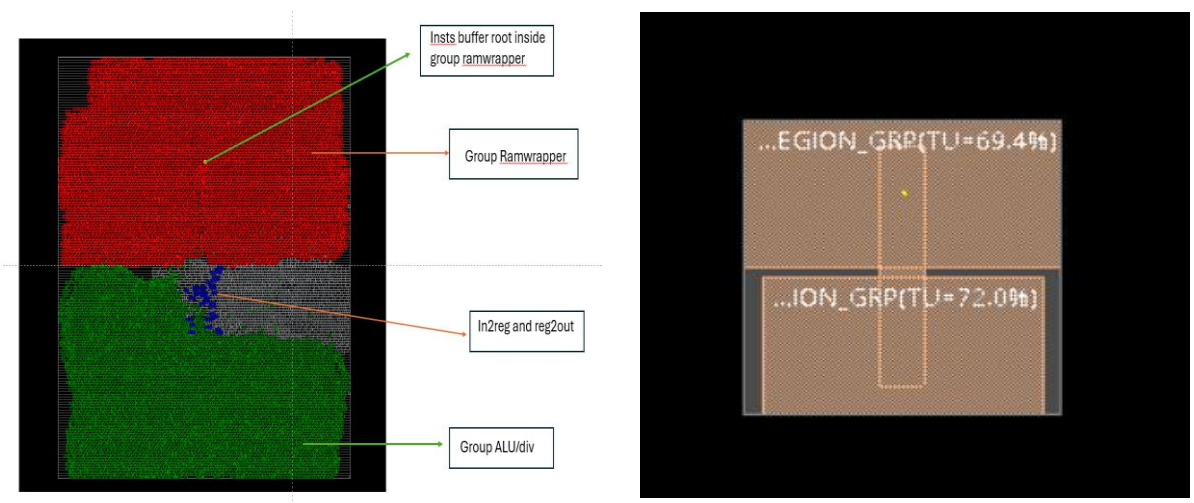
[Insert Figure 8: Highlight of ram_wrapper region is color red, ALU region is color blue]

- in2reg: The in2reg logic was constrained into a vertical central region. This helped keep the input-side registers close to the center of the active logic and reduced excessive spreading during placement.

This grouping method was chosen to improve timing without creating a routing disaster later. Instead of only forcing cells toward the exact center, the placement was guided according to module size, timing role, and physical connectivity.

Instance	Module	Cell Count	Cell Area	Net Area	Total Area
cpu_32bit_top		19608	1097735.648	22505.866	1120241.514
U_CORE	cpu_32bit	19525	67731.886	22147.415	89879.301
DP	cpu_32bit__dp	11821	36244.391	13341.346	49585.737
ACC_reg	reg_w_enable_WIDTH32_223	36	330.723	1.482	332.205
ALU	cpu_32bit__alu	11588	35006.769	13090.678	48097.447
add_74_37	add_unsigned	222	424.678	202.565	627.243
div_128_37	divide_unsigned_rtlopto_model_9	9456	29285.956	10261.919	39547.875
mul_116_37	mult_unsigned	969	3312.432	1293.730	4606.162
sll_94_28	shift_left_vlog_unsigned	158	338.272	179.339	517.612
srl_104_28	shift_right_vlog_unsigned	161	336.519	180.841	517.360
sub_84_37	sub_unsigned	208	372.276	182.076	554.352
IR_reg	reg_w_enable_WIDTH32_224	13	95.427	0.000	95.427
MAR_reg	reg_w_enable_WIDTH7	9	74.223	0.000	74.223
MDR_reg	reg_w_enable_WIDTH32	34	315.333	0.000	315.333
OVERFL_reg	reg_w_enable_WIDTH1_222	2	10.269	0.000	10.269
PC_reg	reg_w_enable_WIDTH7_225	9	62.253	0.000	62.253
ZERO_reg	reg_w_enable_WIDTH1	2	10.269	0.000	10.269
FSM	cpu_32bit__fsm	63	135.141	62.016	197.157
ram_wrapper	CDK_S128x16_wrapper	7641	31352.354	8661.415	40013.769
u_ram128x16_soft	CDK_S128x16_SYNTHESIZE_RAM	7641	31352.354	8661.415	40013.769
U_IO	io_wrapper	83	1030003.762	0.741	1030004.503

[Figure 9: report_area screenshot showing ram_wrapper and ALU/div dominance]



[Figure 10: Highlight of ram_wrapper region, ALU/div region, and central in2reg region]

d. Placement Result and Conclusion

After testing several placement versions, it was observed that the best result did not come from a completely default flow. Better timing behavior was achieved when the design was placed with stronger physical guidance and then optimized incrementally.

In practice, running place_design followed by opt_design more than once gave a better result for this design than relying on a single optimization pass. The reason is that the design contains several large and timing-sensitive logic groups, so one optimization pass was not enough to stabilize both timing and physical structure.

Another important observation was related to clock constraint strategy. During placement, a temporarily tighter clock period of **1.5 ns** was applied to push the tool toward stronger timing-driven optimization. This helped the placement engine focus more aggressively on critical timing paths during the early stage. After

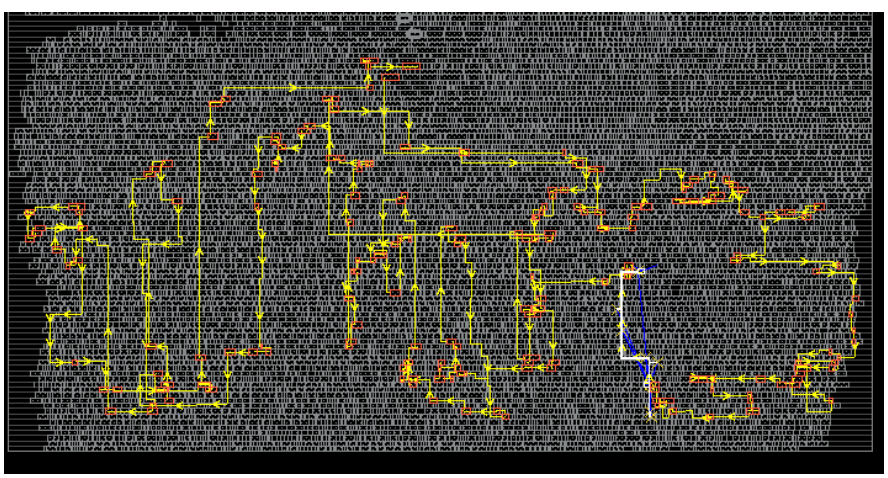
the physical structure became more stable, the clock period was restored to the nominal **2.0 ns** in the later stages to avoid excessive congestion pressure and to maintain a better balance between timing closure and routability.

Overall, the final placement approach was based on the following principles: keep the design compact, keep the root clock buffer near the major sequential region, group the large modules according to timing behavior, and improve timing early without making routing impossible later.

This approach provided a better tradeoff between timing improvement, physical organization, and routability.

```
+--------------------+---------+---------+---------+---------+
|     Setup mode     |   all   | reg2reg |reg2cgate| default |
+--------------------+---------+---------+---------+---------+
|           WNS (ns):| -0.589  | -0.589  |  0.236  |  0.065  |
|           TNS (ns):| -2.289  | -2.289  |  0.000  |  0.000  |
|    Violating Paths:|   20    |   20    |    0    |    0    |
|          All Paths:|  4527   |  4358   |   134   |   68    |
+--------------------+---------+---------+---------+---------+

+----------------+-----------------------------+------------------+
|                |              Real           |       Total      |
|    DRVs        +-----------------+-----------+------------------|
|                |  Nr nets(terms) | Worst Vio |  Nr nets(terms)  |
+----------------+-----------------+-----------+------------------+
|   max_cap      |      0 (0)      |   0.000   |      0 (0)       |
|   max_tran     |      0 (0)      |   0.000   |      0 (0)       |
|   max_fanout   |     33 (33)     |    -9     |     34 (34)      |
|   max_length   |      0 (0)      |    0      |      0 (0)       |
+----------------+-----------------+-----------+------------------+

Density: 87.547%
Routing Overflow: 0.00% H and 0.00% V
```

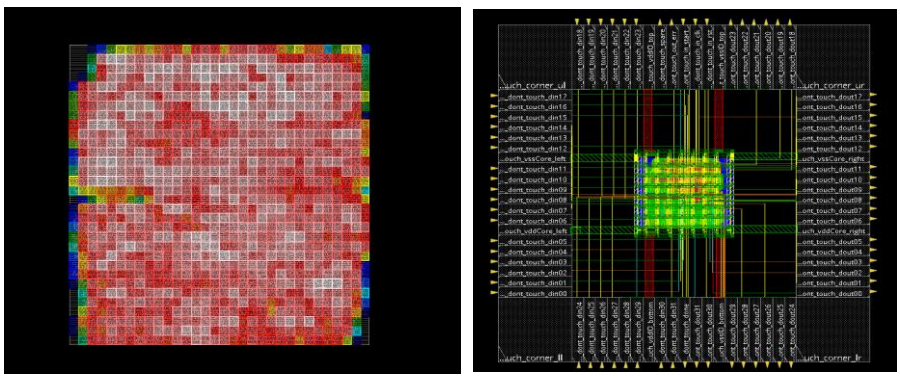
[Figure 11: Placement result after optimization]



[Figure 12:This is the worst negative slack path]

```
Usage: (1.3%H 1.4%V) = (2.012e+05um 2.138e+05um) = (117658 125016)
Overflow: 0 = 0 (0.00% H) + 0 (0.00% V)

Congestion distribution:

Remain  cntH            cntV
-------------------------------------
-------------------------------------
  1:    772      0.14%  0        0.00%
  2:    13       0.00%  777      0.14%
  3:    141560  25.26%  203808  36.36%
  4:    210223  37.51%  147313  26.28%
  5:    207841  37.09%  208593  37.22%
[hotspot] +------------+---------------+---------------+
[hotspot] |            |   max hotspot | total hotspot |
[hotspot] +------------+---------------+---------------+
[hotspot] | normalized |          0.00 |          0.00 |
[hotspot] +------------+---------------+---------------+
```

[Figure 13: Report congestion -hotspot -overflow]



[Insert Figure 14: Density Map & Final placement view]

3. Clock Tree Synthesis (CTS)

a. CTS Preparation and Root Clock Buffer Strategy

After placement, the next stage was Clock Tree Synthesis. In this project, the root clock buffer was kept on purpose because it is required to support clock distribution. Before CTS, the root clock buffer was placed near the center of the ram_wrapper region, because most sequential sinks are concentrated there. This location helps reduce skew risk and keeps the clock path shorter to the main sink region.

The CTS flow also used a defined list of clock cells:

```
cts_buffer_cells = {CLKBUFX4 CLKBUFX6 CLKBUFX8 CLKBUFX12 CLKBUFX16}
cts_inverter_cells = {CLKINVX4 CLKINVX6 CLKINVX8 CLKINVX12 CLKINVX16}
```

This cell list gives enough drive choices for CTS while keeping the clock tree under control.

b. CTS Targets and Timing Analysis Configuration

At the CTS stage, the clock period was returned to the nominal value of 2.0 ns. A tighter clock was used only during placement to push stronger early optimization. For CTS, the nominal clock is more suitable for clock balance, skew control, and insertion delay evaluation.

The main CTS targets in this run were:

```
cts_target_max_transition_time = 0.2
cts_target_skew = 0.1
opt_hold_target_slack = 0.05
opt_setup_target_slack = 0.0
```

OCV and CPPR were enabled to make timing analysis more realistic for both setup and hold checks.

```
set_dont_use [get_lib_cells {*X1 INVX20 INVX16}] true
```

Because the target utilization was 80%, the design was already dense. Therefore, X1 cells and very large inverters were blocked. This helped the tool avoid weak drive cells and also avoid large cells that could increase local congestion.

The design was also allowed to use clock inverters automatically when this was useful for tree construction.

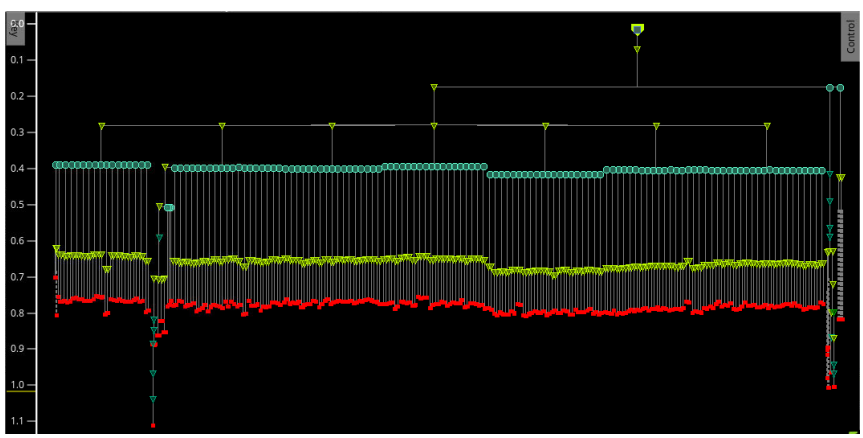
c. Clock Routing Rule and NDR Strategy

The clock routing was not left at the default condition. Separate route rules were prepared for leaf, trunk, and top clock nets.

The route assignment was organized as follows:

```
leaf_rule  : Metal2 to Metal5 using clkroute_1track
trunk_rule : Metal6 to Metal8 using clkroute_2tracks with VSS shielding
top_rule   : Metal9 to Metal11 using clkroute_2tracks with VSS shielding
```

This structure keeps local clock branches on mid layers, while larger clock branches move to upper layers with shielding for cleaner routing. The NDR was used only for clock nets, not for normal signal nets, so clock quality could be improved without adding too much routing congestion.



[Figure 15: Routed clock view after CTS]

d. CTS Execution and Result

After the clock cells, timing targets, and route rules were prepared, the CTS stage was run by the CCOpt flow. A clock tree specification was generated first, and then ccopt_design was used to build and optimize the clock tree.

Additional fanout control was also applied during CTS, including cts_top_fanout_threshold = 40, cts_max_fanout = 20, and design max fanout = 15. These settings helped prevent overly heavy clock branching and kept the tree easier to manage physically.

The routing range for CTS was limited from Metal2 to Metal11. This was done to keep the routing structure consistent with the leaf, trunk, and top route hierarchy.

Overall, the CTS strategy in this design was based on four points: preserve the root clock buffer, place it near the main sink region, use dedicated clock cells and route rules, and build the clock tree under realistic timing settings.

```
+--------------------+---------+---------+---------+---------+---------+---------+---------+
|     Setup mode     |   all   | reg2reg | in2reg  | reg2out | in2out  | default |1:reg..te|
+--------------------+---------+---------+---------+---------+---------+---------+---------+
|           WNS (ns):|  0.003  |  0.003  |  0.467  |  0.046  |   N/A   |  0.261  |  0.038  |
|           TNS (ns):|  0.000  |  0.000  |  0.000  |  0.000  |   N/A   |  0.000  |  0.000  |
|    Violating Paths:|    0    |    0    |    0    |    0    |   N/A   |    0    |    0    |
|          All Paths:|  4527   |  4267   |   33    |   35    |   N/A   |   91    |   134   |
+--------------------+---------+---------+---------+---------+---------+---------+---------+

+----------------+-------------------------------+------------------+
|                |              Real             |       Total      |
|    DRVs        +------------------+------------+------------------|
|                |  Nr nets(terms)  | Worst Vio  |  Nr nets(terms)  |
+----------------+------------------+------------+------------------+
|   max_cap      |      0 (0)       |   0.000    |      0 (0)       |
|   max_tran     |      0 (0)       |   0.000    |      0 (0)       |
|   max_fanout   |      0 (0)       |     0      |      0 (0)       |
|   max_length   |      0 (0)       |     0      |      0 (0)       |
+----------------+------------------+------------+------------------+

Density: 85.473%
```

[Figure 16: ccopt_design result or CTS report]

```
-----------------------------------
```

Even though the command used was route_design -global_detail, this step was treated as the main global-routing stage in this report because it created the first complete routed result after CTS. At this point, the design was checked again by timing and route-related verification commands.

```
check_timing -verbose
check_footprint
```

b. Detail Routing and Post-Route Repair

After the first routing pass, the design entered the detail-route refinement stage. This stage was used to check the routed database more carefully and to repair the remaining physical issues.

Before repair, the design was checked again for placement legality, DRC, antenna, and connectivity. This step was important because the project requires not only completed routing, but also a clean and verifiable final layout.

```
check_place
check_drc
check_antenna
check_connectivity
```

At this stage, the design still showed one hold-time violation after detail routing. The violated path was checked in the GUI and analyzed again with report_timing. It was found that the data path was too fast, so an extra delay had to be added. To fix this problem, a buffer was inserted after the inverter on the data path by using the following ECO command:

eco_add_repeater -pins <U_CORE/DP/ALU/DIV128_37…> -cell BUFF1X -name BUFFQUYET-relative_distance_to_sink 0.5

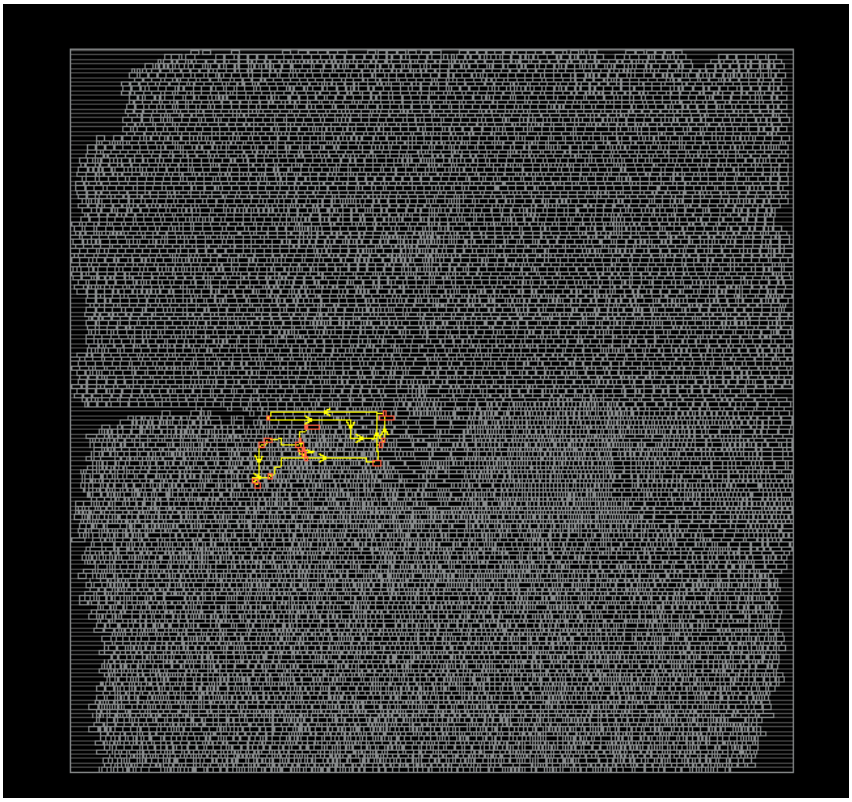
Setup mode	all	reg2reg	in2reg	reg2out	in2out	default	1:reg..te
WNS (ns):	0.050	0.050	0.439	0.060	N/A	0.330	0.133
TNS (ns):	0.000	0.000	0.000	0.000	N/A	0.000	0.000
Violating Paths:	0	0	0	0	N/A	0	0
All Paths:	4527	4267	33	35	N/A	91	134

Hold mode	all	reg2reg	in2reg	reg2out	in2out	default	1:reg..te
WNS (ns):	-0.003	-0.003	0.600	1.407	N/A	0.095	0.370
TNS (ns):	-0.003	-0.003	0.000	0.000	N/A	0.000	0.000
Violating Paths:	1	1	0	0	N/A	0	0
All Paths:	4527	4267	33	35	N/A	91	134

[Figure 18: Post-detail-route hold violation]

```
      Launch Clock:=    9.638
         Data Path:+    1.269
             Slack:=   -0.003
    Timing Path:

#-------------------------------------------------------------------------------------------------------------------------------------------
# Timing Point                                          Flags  Arc          Edge  Cell                            Fanout  Trans   Delay  Arrival
#                                                                                                                          (ns)    (ns)     (ns)
#-------------------------------------------------------------------------------------------------------------------------------------------
  clk_pin                                               -      clk_pin      R     (arrival)                            1  0.004   0.000    9.146
  U_IO/set_dont_touch_in_clk/OUT                        -      PAD->OUT     R     PADI                                 1  0.004   0.056    9.203
  U_IO/set_dont_touch_root_clk_buf/Y                    -      A->Y         R     BUFX8                                3  0.074   0.110    9.313
  U_CORE/DP/MDR_reg/RC_CG_HIER_INST129/RC_CGIC_INST/ECK -      CK->ECK      R     TLATNTSCAX3                          2  0.075   0.237    9.550
  U_CORE/DP/MDR_reg/CTS_ccl_a_buf_00263/Y               -      A->Y         R     CLKBUFX6                            18  0.051   0.087    9.638
  U_CORE/DP/MDR_reg/qout_regx30x/Q                      -      CK->Q        F     DFFRHQX2                             1  0.060   0.278    9.916
  U_CORE/DP/FE_OFC635_mdrx30x/Y                         -      A->Y         R     INVX2                                3  0.033   0.035    9.951
  U_CORE/DP/FE_OFC3067_mdrx30x/Y                        -      A->Y         F     INVX1                                3  0.045   0.057   10.008
  U_CORE/DP/ALU/div_128_37/g162650/Y                    -      B->Y         R     NAND2X1                              2  0.076   0.039   10.047
  U_CORE/DP/ALU/div_128_37/g162600/Y                    -      A->Y         F     INVX1                                3  0.048   0.038   10.085
  U_CORE/DP/ALU/div_128_37/g174584/Y                    -      AN->Y        F     NAND2BX1                             1  0.061   0.095   10.180
  U_CORE/DP/ALU/div_128_37/FE_RC_2094_0/Y               -      B0->Y        R     OAI21X1                              1  0.065   0.042   10.222
  U_CORE/DP/ALU/div_128_37/g151570__3680/Y              -      B0->Y        F     OAI21X1                              1  0.045   0.097   10.319
  U_CORE/DP/ALU/div_128_37/g151528__7410/Y              -      B0->Y        R     OAI21X4                              1  0.132   0.064   10.383
  U_CORE/DP/ALU/div_128_37/g151515__8246/Y              -      B0->Y        F     AOI21X4                              1  0.058   0.037   10.421
  U_CORE/DP/ALU/div_128_37/FE_RC_4979_0/Y               -      B->Y         R     NOR2X4                               1  0.035   0.042   10.463
  U_CORE/DP/ALU/div_128_37/g874/Y                       -      A->Y         F     NOR2X4                               1  0.058   0.042   10.504
  U_CORE/DP/ALU/div_128_37/g870/Y                       -      B->Y         R     NAND2X4                              2  0.034   0.030   10.534
  U_CORE/DP/ALU/div_128_37/QUOTIENT[1]                  -      QUOTIENT[1]  R     divide_unsigned_rtlopto_model_9      -      -       -   10.534
  U_CORE/DP/ALU/FE_OCPC3819_n_552/Y                     -      A->Y         R     BUFX3                                1  0.034   0.074   10.608
  U_CORE/DP/ALU/g11576/Y                                -      A1->Y        F     AOI21X4                              2  0.045   0.064   10.673
  U_CORE/DP/ALU/FE_PSC4860_n_1535/Y                     -      A->Y         F     BUFX2                                1  0.036   0.081   10.754
  U_CORE/DP/ALU/FE_PHC4903_n_1535/Y                     -      A->Y         F     BUFX2                                1  0.044   0.086   10.840
  U_CORE/DP/ALU/FE_RC_53_0/Y                            -      A->Y         R     NAND2X2                              1  0.056   0.039   10.879
  U_CORE/DP/ALU/FE_RC_51_0/Y                            -      B->Y         F     NOR2X2                               1  0.028   0.028   10.907
  U_CORE/DP/ZERO_reg/qout_regx0x/SI                     -      SI           F     SDFFRHQX1                            1  0.029   0.000   10.907
#-------------------------------------------------------------------------------------------------------------------------------------------
     Other End Path:

#-----------------------------------------------------------------------------------------------------------
# Timing Point                                  Flags  Arc       Edge  Cell       Fanout  Trans   Delay  Arrival
#                                                                                          (ns)    (ns)     (ns)
#-----------------------------------------------------------------------------------------------------------
  clk_pin                                       -      clk_pin   R     (arrival)       1  0.004   0.000    9.146
  U_IO/set_dont_touch_in_clk/OUT                -      PAD->OUT  R     PADI            1  0.004   0.056    9.203
  U_IO/set_dont_touch_root_clk_buf/Y            -      A->Y      R     BUFX8           3  0.074   0.110    9.313
  U_CORE/CTS_ccl_a_buf_00279/Y                  -      A->Y      R     CLKBUFX16       1  0.075   0.080    9.393
  U_CORE/FE_USKC3207_CTS_5/Y                    -      A->Y      R     CLKBUFX12       1  0.024   0.062    9.456
  U_CORE/FE_USKC3231_CTS_5/Y                    -      A->Y      R     CLKBUFX4        1  0.024   0.072    9.527
```

```
#-------------------------------------------------------------------------------------------------------------------------------------------
# Timing Point                                  Flags  Arc          Edge  Cell                            Fanout  Trans   Delay  Arrival
#                                                                                                                  (ns)    (ns)     (ns)
#-------------------------------------------------------------------------------------------------------------------------------------------
  U_CORE/DP/MDR_reg/qout_regx30x/CK             -      CK           R     (arrival)                          18  0.060       -    9.638
  U_CORE/DP/MDR_reg/qout_regx30x/Q              -      CK->Q        F     DFFRHQX2                            1  0.060   0.278    9.916
  U_CORE/DP/FE_OFC635_mdrx30x/Y                 -      A->Y         R     INVX2                               3  0.033   0.035    9.951
  U_CORE/DP/FE_OFC3067_mdrx30x/Y                -      A->Y         F     INVX1                               3  0.045   0.057   10.008
  U_CORE/DP/ALU/div_128_37/g162650/Y            -      B->Y         R     NAND2X1                             2  0.076   0.052   10.060
  U_CORE/DP/ALU/div_128_37/g162600/Y            <<<    A->Y         F     INVX1                               1  0.050   0.032   10.092
  U_CORE/DP/ALU/div_128_37/quyet123/Y           -      A->Y         F     BUFX2                               3  0.022   0.074   10.166
  U_CORE/DP/ALU/div_128_37/g174584/Y            -      AN->Y        F     NAND2BX1                            1  0.039   0.089   10.255
  U_CORE/DP/ALU/div_128_37/FE_RC_2094_0/Y       -      B0->Y        R     OAI21X1                             1  0.065   0.042   10.298
  U_CORE/DP/ALU/div_128_37/g151570__3680/Y      -      B0->Y        F     OAI21X1                             1  0.045   0.097   10.395
  U_CORE/DP/ALU/div_128_37/g151528__7410/Y      -      B0->Y        R     OAI21X4                             1  0.132   0.064   10.459
  U_CORE/DP/ALU/div_128_37/g151515__8246/Y      -      B0->Y        F     AOI21X4                             1  0.058   0.037   10.496
  U_CORE/DP/ALU/div_128_37/FE_RC_4979_0/Y       -      B->Y         R     NOR2X4                              1  0.035   0.042   10.538
  U_CORE/DP/ALU/div_128_37/g874/Y               -      A->Y         F     NOR2X4                              1  0.058   0.042   10.580
  U_CORE/DP/ALU/div_128_37/g870/Y               -      B->Y         R     NAND2X4                             2  0.034   0.030   10.610
  U_CORE/DP/ALU/div_128_37/QUOTIENT[1]          -      QUOTIENT[1]  R     divide_unsigned_rtlopto_model_9     -      -       -   10.610
  U_CORE/DP/ALU/FE_OCPC3819_n_552/Y             -      A->Y         R     BUFX3                               1  0.034   0.074   10.684
  U_CORE/DP/ALU/g11576/Y                        -      A1->Y        F     AOI21X4                             2  0.045   0.064   10.748
  U_CORE/DP/ALU/FE_PSC4860_n_1535/Y             -      A->Y         F     BUFX2                               1  0.036   0.081   10.830
  U_CORE/DP/ALU/FE_PHC4903_n_1535/Y             -      A->Y         F     BUFX2                               1  0.044   0.086   10.916
  U_CORE/DP/ALU/FE_RC_53_0/Y                    -      A->Y         R     NAND2X2                             1  0.056   0.039   10.955
  U_CORE/DP/ALU/FE_RC_51_0/Y                    -      B->Y         F     NOR2X2                              1  0.028   0.028   10.983
  U_CORE/DP/ZERO_reg/qout_regx0x/SI             -      SI           F     SDFFRHQX1                           1  0.029   0.000   10.983
#-------------------------------------------------------------------------------------------------------------------------------------------

@innovus 13> place_det
```



[Figure 19: Data path violation hold time]

```
+--------------------+--------------------------------------------------+
|     ID Number      |                  Path Group Name                 |
+--------------------+--------------------------------------------------+
|         1          |reg2ckgate                                        |
+--------------------+--------------------------------------------------+

+--------------------+---------+---------+---------+---------+---------+---------+---------+
|     Setup mode     |   all   | reg2reg | in2reg  | reg2out | in2out  | default |1:reg..te|
+--------------------+---------+---------+---------+---------+---------+---------+---------+
|           WNS (ns):|  0.050  |  0.050  |  0.439  |  0.060  |   N/A   |  0.330  |  0.133  |
|           TNS (ns):|  0.000  |  0.000  |  0.000  |  0.000  |   N/A   |  0.000  |  0.000  |
|    Violating Paths:|    0    |    0    |    0    |    0    |   N/A   |    0    |    0    |
|          All Paths:|  4527   |  4267   |   33    |   35    |   N/A   |   91    |   134   |
+--------------------+---------+---------+---------+---------+---------+---------+---------+

+----------------+-----------------------------+------------------+
|                |              Real           |       Total      |
|    DRVs        +-----------------+-----------+------------------|
|                |  Nr nets(terms) | Worst Vio |  Nr nets(terms)  |
+----------------+-----------------+-----------+------------------+
|   max_cap      |      0 (0)      |   0.000   |      0 (0)       |
|   max_tran     |      0 (0)      |   0.000   |      0 (0)       |
|   max_fanout   |      0 (0)      |     0     |      0 (0)       |
|   max_length   |      0 (0)      |     0     |      0 (0)       |
+----------------+-----------------+-----------+------------------+

Density: 85.705%
```

[Figure 20: meet timing setup time and holdtime]

c. Antenna Repair

The first full DRC check after detail route still reported remaining violations. To fix them in a structured way, an ECO-based repair flow was used. Antenna repair was enabled, diode insertion was allowed, and ECO routing was applied to clean the local problem areas.

```
set_db route_detail_fix_antenna true
set_db route_antenna_diode_insertion true
route_eco
```

After antenna diode insertion, the power connections were restored, and the design was checked again. This repeated re-check is important because the teacher requires DRC debug to be done step by step, without damaging the parts that were already clean.

```
connect_global_net VDD -type pgpin -pin VDD -inst *
connect_global_net VSS -type pgpin -pin VSS -inst *
check_drc
check_antenna
check_connectivity
```

In this design, antenna behavior was also reviewed carefully. The violation was related to long metal area on the net. The solution was to reduce the effective net area during the repair stage so that the antenna condition could be removed without creating new routing problems.

d. Post-Route Optimization and Final Result

After the main physical repairs were completed, post-route optimization was run for both setup and hold. This final step was used to improve timing while keeping the routed structure stable.

```
opt_design -post_route -setup -hold
```

Final timing and constraint reports were generated after the repair flow. The design was then checked again for DRC, antenna, and connectivity. The final result was a routed layout with clean physical quality, clean connectivity, and a condition that could proceed to the final stage.

Overall, the routing flow in this project was based on two parts: first, create the initial routed structure after CTS; second, refine and repair the design through detail-route checks, ECO routing, antenna repair, and post-route optimization. This flow matched the project requirement of completing both routing stages and converging to a clean final implementation.

4. Fix DRC and antenna ratio area

a. DRC Fix

After routing, the design still had two DRC violations. Both violations were short violations on Metal1.

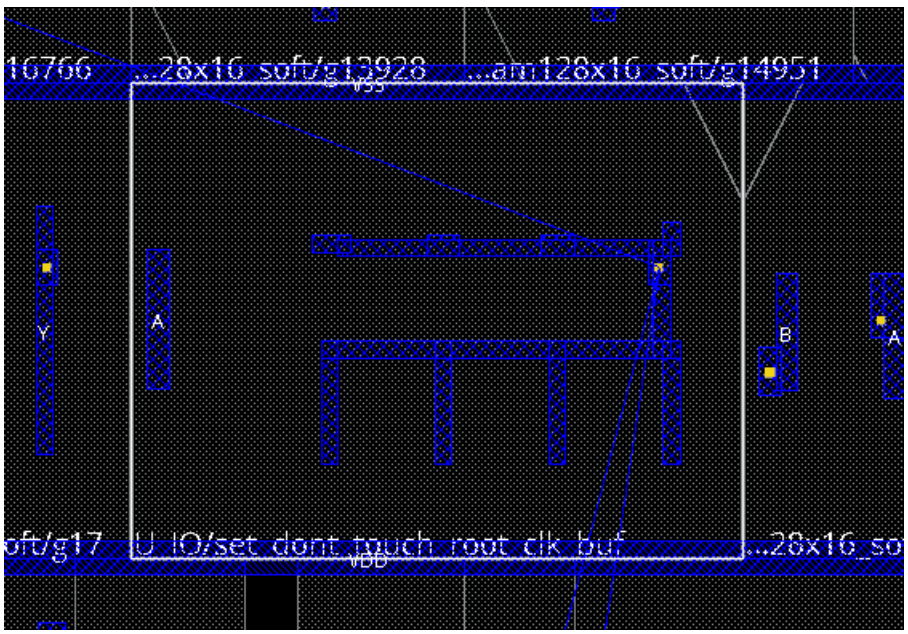
After checking the violation area, the root cause was found at the root clock buffer. When this cell was placed, its orientation was set to R0, so its power pins were not aligned correctly with the local VDD and VSS rails. Because of that, a short violation was created.

To fix this problem, the orientation of the root clock buffer was changed from R0 to R180. After rotating the cell, the power pins matched the local power rails correctly, and the short violations were removed.

This fix was simple, but it was important because it showed that some DRC violations were caused not by routing complexity, but by an incorrect physical orientation of a critical cell.

```
 Verification Complete : 2 Viols.

Violation Summary By Layer and Type:

                 Short   Totals
       Metal1        2        2
       Totals        2        2

*** End Verify DRC (CPU TIME: 0:01:58  ELAPSED TIME: 0:02:03  MEM: 264.1M) ***

1
```

[Insert Figure 21:two violation DRC]



[Insert Figure 22: Root clock buffer after R180 orientation fix]

b. Antenna Ratio Area

Another issue observed in the final stage was an antenna ratio area violation.

In physical design, an antenna violation happens when a metal net becomes too large before it is safely connected to a diffusion or discharge path. During manufacturing, this large metal area can collect too much charge. If the charge is too high, it may damage the thin gate oxide of the connected cell pin.

In this design, the antenna problem was related to a routed net with too much metal area. Two repair methods were considered.

Solution 1:

```
check_antenna

set net [get_db [get_db markers -if {.subtype == "Antenna_Area_Ratio"}] .objects.name ]
```

```
delete_nets $net
route_global_detail
```

Delete the violating net and route it again.

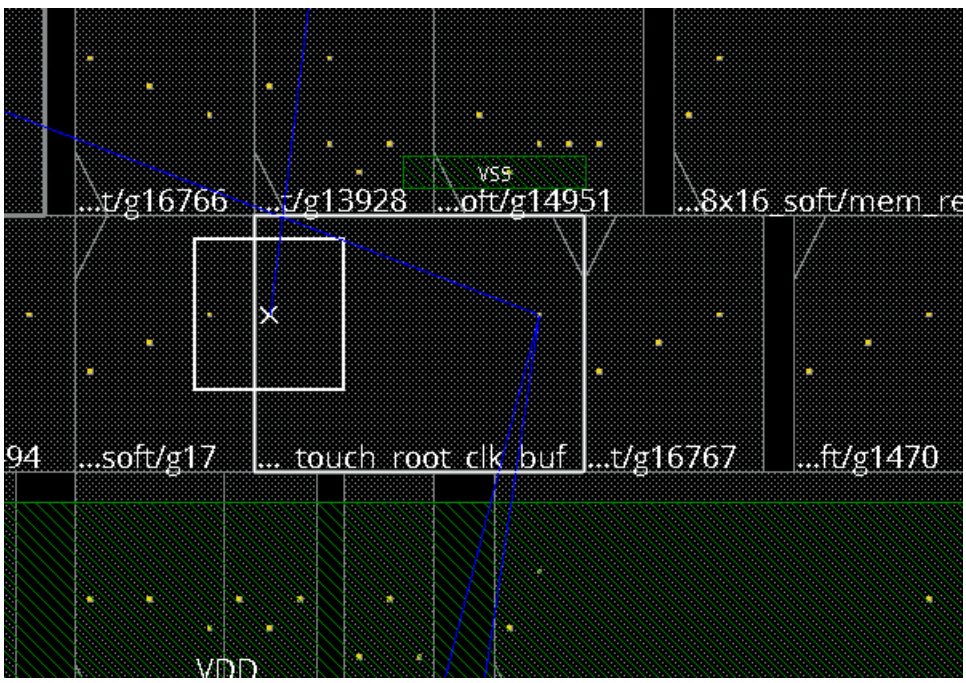
This method allows the router to rebuild the connection with a different path and possibly reduce the antenna area.

Solution 2:

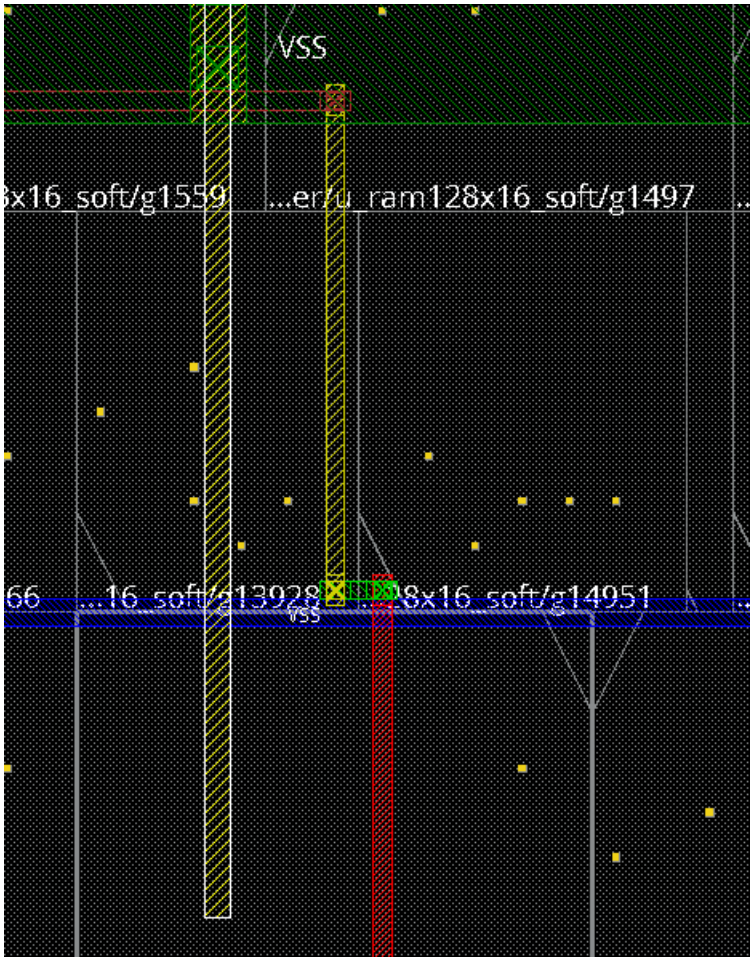
Reduce the width of the violating net.

By reducing the wire width, the total metal area of the net is also reduced. As a result, the stored charge becomes smaller, and the antenna ratio can return to a legal value.

Both solutions follow the same physical idea: reduce the effective metal area of the violating net so that it will not store too much charge and damage the gate pin.



[Insert Figure 23: Antenna ratio area violation]



[Insert Figure 24: Net after reroute or width reduction]

c. Final Result

After fixing the root clock buffer orientation and repairing the antenna violation, the design was checked again. The final result showed that the routing became clean and the design could continue to the final stage.

This repair process followed the project requirement of debugging physical violations in a structured way and re-checking the design after each fix until the layout became clean.

```
 VERIFY DRC ...... Sub-Area: {929.280 1223.040 1013.760 1300.000} 222 of 225
 VERIFY DRC ...... Sub-Area : 222 complete 0 Viols.
 VERIFY DRC ...... Sub-Area: {1013.760 1223.040 1098.240 1300.000} 223 of 225
 VERIFY DRC ...... Sub-Area : 223 complete 0 Viols.
 VERIFY DRC ...... Sub-Area: {1098.240 1223.040 1182.720 1300.000} 224 of 225
 VERIFY DRC ...... Sub-Area : 224 complete 0 Viols.
 VERIFY DRC ...... Sub-Area: {1182.720 1223.040 1260.000 1300.000} 225 of 225
 VERIFY DRC ...... Sub-Area : 225 complete 0 Viols.

 Verification Complete : 0 Viols.

*** End Verify DRC (CPU TIME: 0:01:47  ELAPSED TIME: 0:01:50  MEM: 264.1M) ***
```

[Figure 25: Final clean DRC result]

```
@innovus 29> check_connectivity
VERIFY_CONNECTIVITY use new engine.

******** Start: VERIFY CONNECTIVITY ********
Start Time: Wed Apr 15 09:35:19 2026

Design Name: cpu_32bit_top
Database Units: 2000
Design Boundary: (0.0000, 0.0000) (1260.0000, 1300.0000)
Error Limit = 1000; Warning Limit = 50
Check all nets
**** 09:35:19 **** Processed 5000 nets.
**** 09:35:20 **** Processed 10000 nets.
**** 09:35:20 **** Processed 15000 nets.
**** 09:35:20 **** Processed 20000 nets.

Begin Summary
  Found no problems or warnings.
End Summary

End Time: Wed Apr 15 09:35:20 2026
Time Elapsed: 0:00:01.0

******** End: VERIFY CONNECTIVITY ********
  Verification Complete : 0 Viols.  0 Wrngs.
  (CPU Time: 0:00:01.2  MEM: 0.000M)
```

[Figure 26: Final clean connectivity]